

DevSecOps 4-3-Intro-to-Kubernetes

Task 1 - Introduction

Kubernetes (or K8s) has become a cornerstone of DevSecOps by enabling automated container orchestration and scalable security controls across modern application deployment pipelines.

Task 2 - Kubernetes 101

Organizations transitioned from monolithic applications to microservices architectures to independently scale specific business functions under heavy traffic demand. The rise of microservices created massive fleets of containers that required dedicated technology to manage and organize them at scale.

Kubernetes functions as a container orchestration system, acting like a conductor that dynamically creates, removes, and balances traffic across containers based on application demand.

The Many Benefits of Kubernetes:

- Kubernetes ensures high availability and performance by automatically load balancing, removing single points of failure, and scaling application resources dynamically based on traffic demand.
- Kubernetes is also **highly portable** in that it can be run anywhere with nearly any type of infrastructure and can be used with a single or multi-cloud set, making it a very flexible tool.
- Another benefit of Kubernetes is its popularity; because of this, many technologies are compatible with it and can be used to make each tool more powerful.

Answer the questions below

Which benefit of Kubernetes means that it can run anywhere on any type of infrastructure?

highly portable

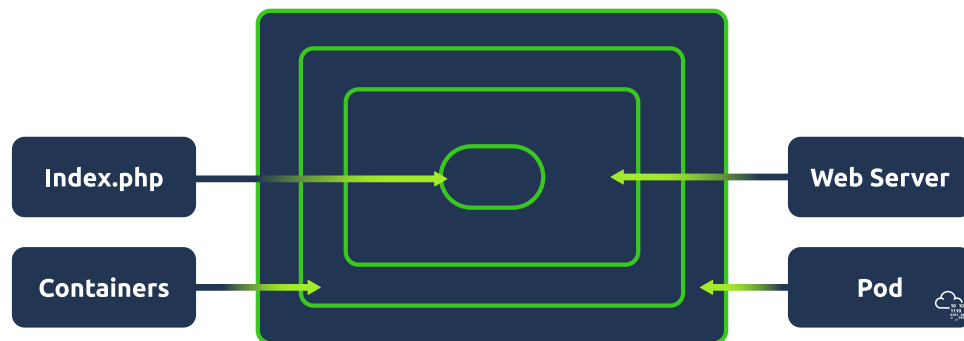
Fill in the blank "Kubernetes is a _____ system".

container orchestration

Task 3 - Kubernetes Architecture

Cluster Architecture: Understanding Kubernetes architecture involves breaking down its core structural components to see how they coordinate and connect.

Kubernetes Pod: Pods are the fundamental deployment unit in Kubernetes, grouping one or more containers that share network and storage resources while scaling together as a single replica.



Kubernetes Nodes: Nodes are virtual or physical machines—divided into control plane and worker types—that provide the essential execution environment and services for running application pods.

Kubernetes Cluster: Simply, a Cluster is just a set of nodes.

The Kubernetes Control Plane: The control plane orchestrates the cluster by managing worker nodes, assigning workloads, and maintaining overall state through specialized architectural components.

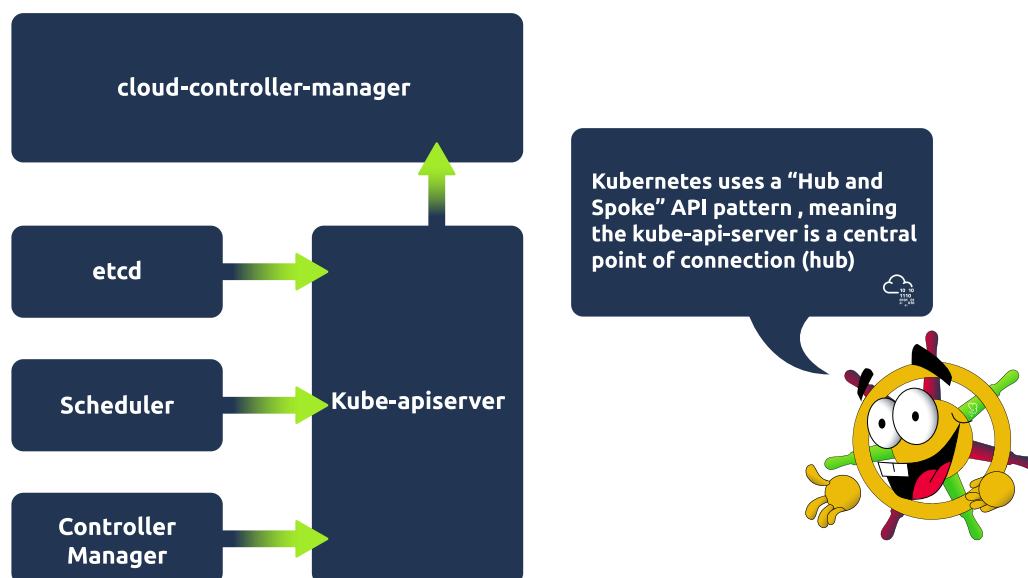
Kube-apiserver: The `kube-apiserver` acts as the scalable front-end entry point for all control plane interactions, exposing the Kubernetes API to users and internal systems.

Etcd: `etcd` serves as a highly available, consistent key-value datastore that holds the authoritative state and configuration data of the entire cluster.

Kube-scheduler: The `kube-scheduler` constantly monitors unassigned pods and selects optimal worker nodes for them based on resource availability and constraints.

Kube-controller-manager: The `kube-controller-manager` runs core controller loops that continuously monitor cluster state, such as detecting offline nodes and triggering corrective actions.

Cloud-controller-manager: The `cloud-controller-manager` abstracts cloud-provider logic, allowing Kubernetes to integrate natively with external cloud infrastructure APIs while keeping core cluster operations decoupled.

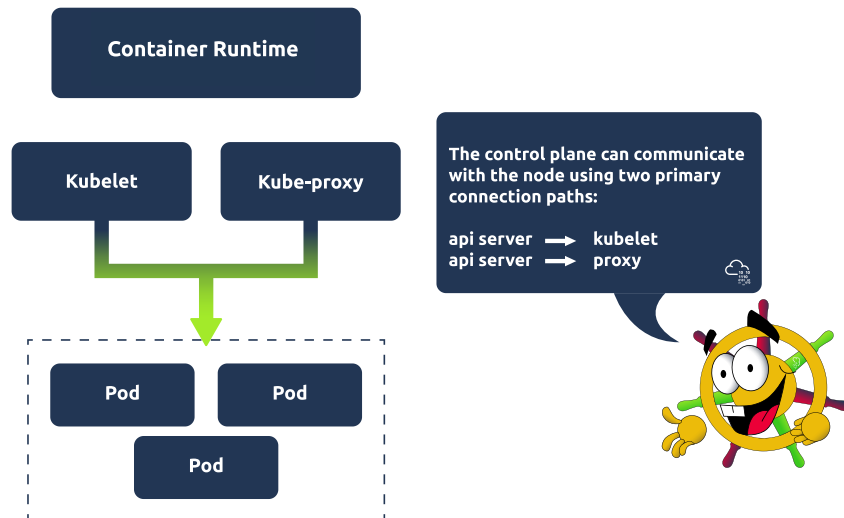


Kubernetes Worker Node: Worker nodes host and manage active application pods through specialized runtime and networking services deployed across every node in the cluster.

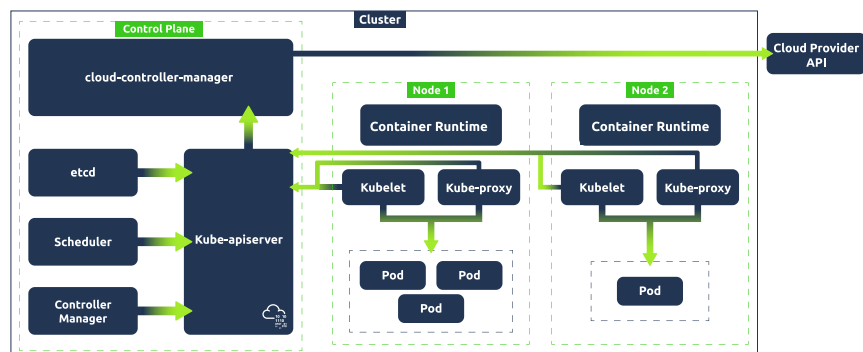
Kubelet: `kubelet` runs as a local agent on every node to interpret pod specifications and enforce that all specified containers remain running and healthy.

Kube-proxy: `kube-proxy` handles internal and external cluster networking by maintaining proxy rules that route traffic through Kubernetes Services to target pods.

Container runtime: The container runtime provides the underlying execution engine on each node required to pull images and run containerized workloads.



Communication Between Components: The Kubernetes architecture functions as a unified cluster, where the control plane components manage state and schedule workloads across physical or virtual nodes, while worker nodes run the pods containing isolated application containers.



Answer the questions below

What is the smallest deployable unit of computing you can create in Kubernetes?

pod

Which control plane component is a key/value store which contains data pertaining to the cluster and its current state?

etcd

Which worker node component is responsible for network communication within the cluster?

kube-proxy

Task 4 - Kubernetes Landscape

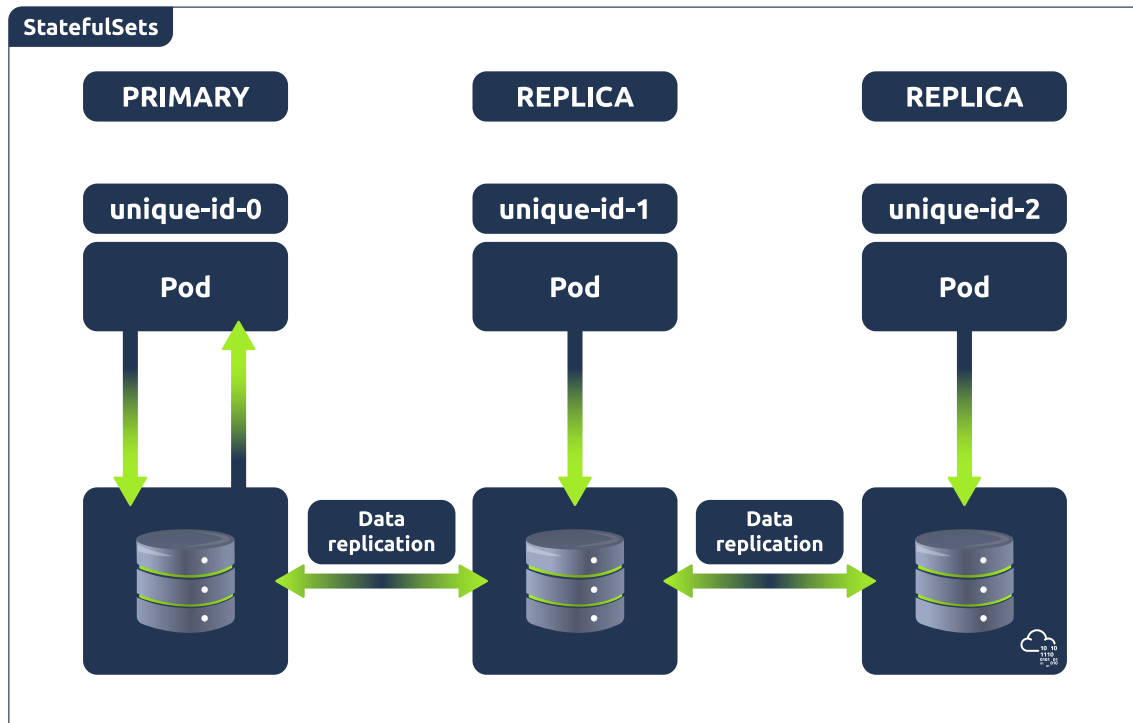
Essential Kubernetes concepts for daily DevSecOps workflows.

Namespaces: Namespaces provide logical isolation within a shared Kubernetes cluster, allowing resources to be organized by tenant or environment with distinct naming scopes.

ReplicaSet: ReplicaSets ensure high availability by maintaining a specified count of identical running pod replicas, though they are usually managed indirectly via Deployments.

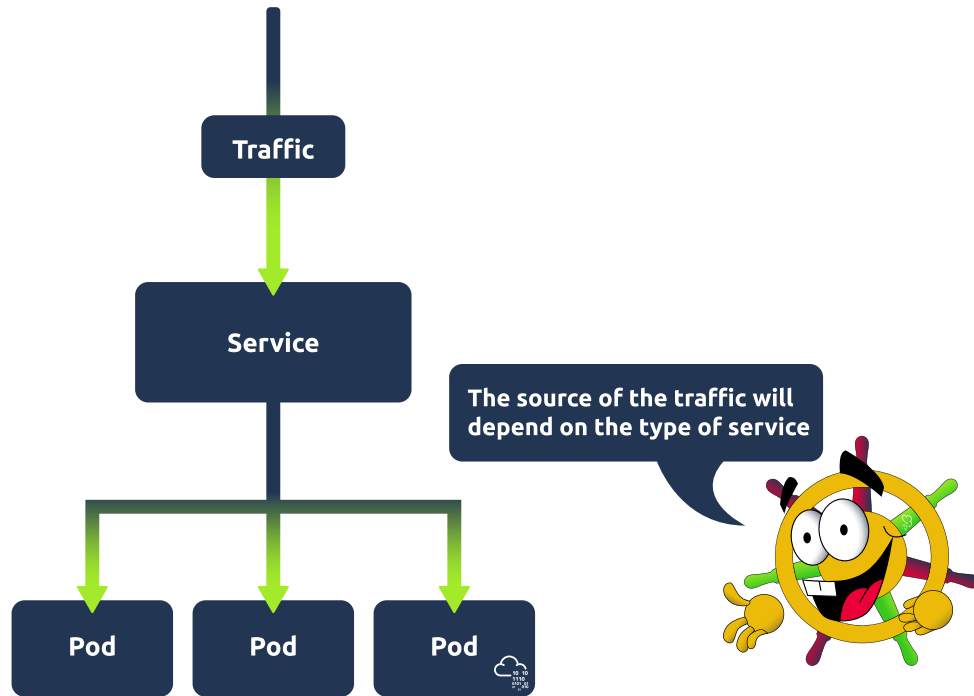
Deployments: Deployments offer declarative state management, allowing developers to define desired pod and ReplicaSet configurations that Kubernetes automatically creates and maintains.

StatefulSets: StatefulSets maintain data consistency during scaling and updates by assigning each pod a persistent identity, strict creation order, and synchronized storage bindings. StatefulSets protect multi-pod databases from data corruption by enforcing persistent network identities, ordered provisioning, and dedicated storage bindings for stateful applications.

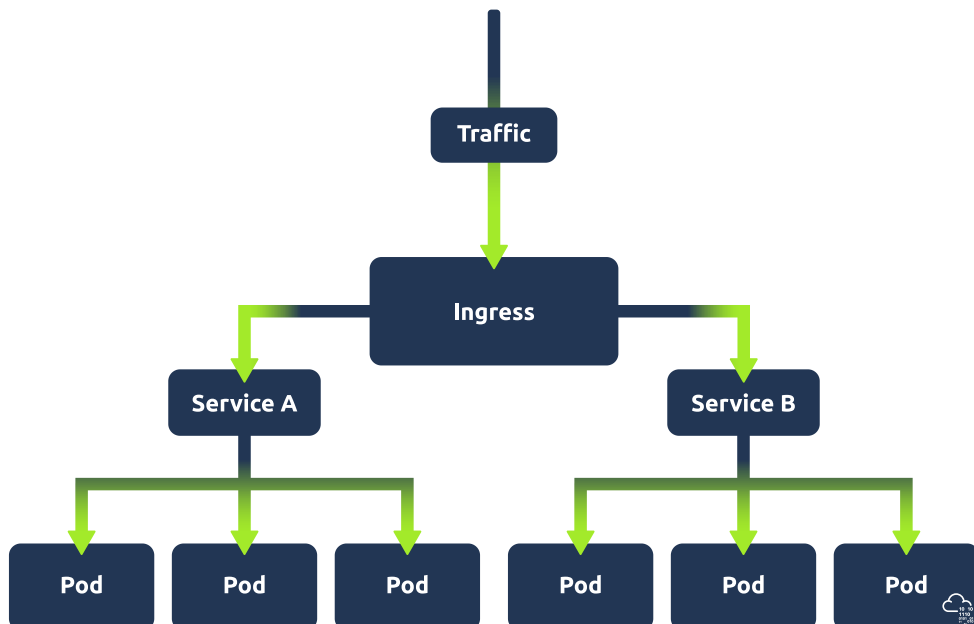


Services: Kubernetes pods are ephemeral and frequently recreated, which creates networking challenges for applications needing consistent communication with them.

Services solve dynamic IP issues by exposing a single static access point to load-balance traffic across pod replicas via types like ClusterIP, NodePort, LoadBalancer, and ExternalName.



Ingress: Ingress serves as a unified entry point and routing manager, directing external user requests to the appropriate backend service based on defined rules.



DevOps vs. DevSecOps in K8s : DevOps focuses on building and maintaining cluster resources, while DevSecOps specializes in integrating security controls

across the Kubernetes lifecycle. Mastering foundational DevOps cluster management is a prerequisite for DevSecOps professionals to effectively harden and secure Kubernetes environments.

Answer the questions below

Which Kubernetes component exposes pods and serves as an access point?

service

Which Kubernetes component can guarantee the availability of X number of pods?

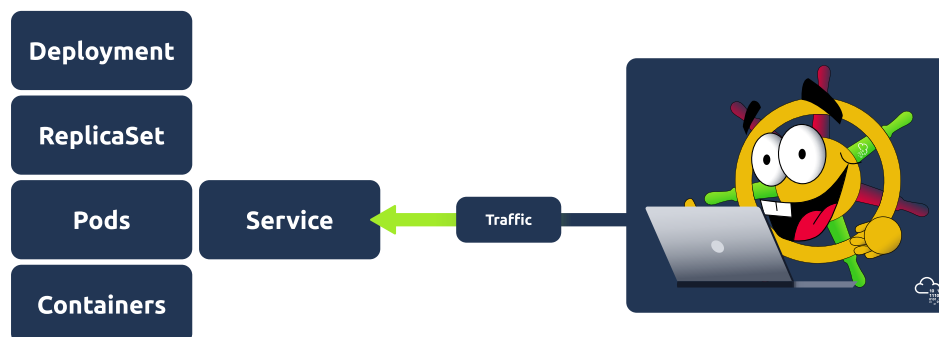
replicaset

What Kubernetes component is used to define a desired state?

deployments

Task 5 - Kubernetes Configuration

This walkthrough illustrates how Deployments, ReplicaSets, Pods, and Services integrate into a cohesive, multi-layered Kubernetes configuration.



Configuring a Deployment and its corresponding Service requires two separate configuration files, typically written in human-readable YAML rather than JSON as a best-practice standard. Every Kubernetes manifest shares a uniform top-level structure consisting of key elements—`apiVersion`, `kind`, `metadata`, and `spec`—to define the target resource and its desired state.

Required Fields: Next, let's discuss the four fields which must be present in each of the YAML files, breaking down what will need to be included in each one:

apiVersion: The version of the Kubernetes API you are going to use to create this object. The API version you use will depend on the object being defined. A cheatsheet for what API version to use for which object can be found [here](#)(opens in new tab).

kind: What kind of object you are going to create (e.g. Deployment, Service, StatefulSet).

metadata: This will contain data that can be used to uniquely identify the object (including name and an optional namespace).

spec: The desired state of the object (for deployment, this might be 3 nginx pods).

Configuring Resources: Services should be defined before backend Deployments so Kubernetes can automatically inject Service environment variables into newly started container Pods. Here is what our example-service.yaml file looks like:

```
apiVersion: v1
kind: Service
metadata:
  name: example-nginx-service
spec:
  selector:
    app: nginx
  ports:
    - protocol: TCP
      port: 8080
      targetPort: 80
  type: ClusterIP
```

Let's break that down: apiVersion: This manifest defines a default ClusterIP Service ("example-nginx-service") using API `v1` to expose port 80 and forward traffic to `targetPort` 80 on Pods matching the `app: nginx` label selector. Now let's take a look at the Deployment YAML and define the back end which this service will point to:

```

apiVersion: apps/v1
kind: Deployment
metadata:
  name: example-nginx-deployment
spec:
  replicas: 3
  selector:
    matchLabels:
      app: nginx
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
      - name: nginx
        image: nginx:latest
        ports:
        - containerPort: 80

```

The Deployment manifest uses a nested `template` spec to configure 3 identical Pods with port 80 and matching `app: nginx` labels, aligning directly with the Service's selector and target port to expose the managed ReplicaSet. Kubernetes continuously compares the desired state declared in YAML config files with the cluster's actual state stored in etcd, automatically triggering corrective actions if a discrepancy occurs.

Answer the questions below

In a config file, you have just declared that you want 4 nginx pods. In which one of the 'required fields' has this been declared?

spec

The configuration file is for a deployment. In which one of the 'required fields' is this declared?

kind

The pods in this deployment will be exposed by a service. In the service configuration file, the target port was set to 80. What should you put as the 'containerPort'?

80

Task 6 - Kubectl

Kubectl to the Rescue: `kubectl` is an essential command-line tool that enables DevSecOps engineers to communicate with the Kubernetes API server, apply YAML configurations, and manage cluster resources.

Kubectl apply: The `kubectl apply` command submits YAML manifest files to the Kubernetes API server, transforming declared desired configurations into active, running cluster resources.

```
kubectl apply -f example-deployment.yaml
```

Kubectl get: The `kubectl get` command inspects the current operational state and health of specified Kubernetes resources within a given namespace. For example, to check the state of a deployment, you would use:

```
kubectl get pods -n example-namespace
```

The output for this command would look something like this:

```
user@tryhackme$ kubectl get pods -n example-namespace
NAME          READY   STATUS    RESTARTS   AGE
example-pod   1/1     Running   0           2m18s
```

As mentioned, this command can be used to check on a variety of resources such as deployments, services, pods, and ReplicaSets.

Kubectl describe: The `kubectl describe` command retrieves comprehensive diagnostic metadata, events, and operational details for cluster resources to assist in troubleshooting and root-cause analysis.

```
kubectl describe pod example-pod -n example-namespace
```

This would give you back some details about the erroring pod. For example:

```
user@tryhackme$ kubectl describe pod example-pod -n example-namespace
```

```
Name:          example-pod
Namespace:     example-namespace
Priority:       0
Service Account: default
Node:          minikube/192.168.49.2
Start Time:    Mon, 22 Jan 2024 14:01:14 +0000
Labels:        <none>
Annotations:   <none>
Status:        Running
IP:            10.244.0.21
...
...
```

You can see here that the describe command details contain some "events". These events can help shine a light on what the issue is. For more details surrounding this event, Kubernetes captures logs from each container in a running pod. These logs can be viewed using our next command.

Kubectl logs: The `kubectl logs` command streams application output and runtime log events from a container inside a Pod, helping developers diagnose specific code-level errors and execution issues.

```
kubectl logs example-pod -n example-namespace
```

These logs can provide valuable insight not just for troubleshooting but for security analysis as well.

Kubectl exec: The `kubectl exec` command opens an interactive terminal session inside a running container using the `-it` flag, allowing engineers to directly

inspect and troubleshoot container environments.

```
kubectl exec -it example-pod -n example-namespace -- sh
```

From here, you can run any command you want from within the container itself. Maybe you want to snoop around for your security analysis or run a command to test connectivity from the container.

Kubectl port-forward: The `kubectl port-forward` command creates a secure network tunnel between a local machine port and a target cluster resource, allowing engineers to directly access and test internal applications via localhost. For example, matching the target port (the port the service is exposed on, which in our configuration example was 8080) to local port 8090 would make this web application accessible on our local machine at `http://localhost:8090`. The resources specified are `resource-type/resource-name`. This would be done using the kubectl port-forward command with the following syntax :

```
kubectl port-forward service/example-service 8090:8080
```

There are, of course, plenty more kubectl commands that can be used to navigate and interact with a Kubernetes cluster, but the ones covered give you a solid foundation, and you can perform some day-to-day DevSecOps tasks using only these.

What command would follow 'kubectl' if you wanted to....

Answer the questions below

| ...troubleshoot a pod by gathering some details about it?

describe

| ...access the container's shell?

exec

| ...check the status of running pods?

get

| ...turn a defined configuration (YAML file) into a running process?

apply

Task 7 - Kubernetes & DevSecOps

As a DevSecOps engineer, you leverage Kubernetes features, security policies, and automation tools to harden workloads, enforce zero-trust networking, and maintain continuous threat visibility across the cluster lifecycle.

Kubernetes and Security: Kubernetes is a rapidly adopted, emerging technology whose widespread integration introduces new security risks into modern software stacks.

Introducing any new tool expands the attack surface, and Kubernetes' default behavior of allowing unrestricted pod-to-pod communication creates significant security risks that DevSecOps engineers must secure.

Kubernetes Hardening: Container hardening is the DevSecOps practice of scanning cluster environments for CVEs and remediating vulnerabilities to minimize breach risks.

Kubernetes hardening fortifies cluster channels and reduces exposure by implementing established security best practices defined by industry and government standards.

Secure your Pods: Some best practices for pod security include:

- Containers that run applications should not have root privileges
- Containers should have an immutable filesystem, meaning they cannot be altered or added to (depending on the purpose of the container, this may not be possible)
- Container images should be frequently scanned for vulnerabilities or misconfigurations
- Privileged containers should be prevented

- [Pod Security Standards](#)(opens in new tab) and [Pod Security Admission](#)(opens in new tab).

Hardening and Separation of your Network:

- Access to the control plane node should be restricted using a firewall and role-based access control in an isolated network
- Control plane components should communicate using Transport Layer Security (TLS) certificates
- An explicit deny policy should be created
- Credentials and sensitive information should not be stored as plain text in configuration files. Instead, they should be encrypted and in Kubernetes secrets

Using Authentication and Authorisation Optimally: Here are some best practices which can help make sure you are making efficient use of Kubernetes authentication and authorisation features:

- Anonymous access should be disabled
- Strong user authentication should be used
- RBAC policies should be created for the various teams using the cluster and the service accounts utilised

Keeping an Eye Out: Here are some logging best practices to ensure you know exactly what is going on in your cluster and can detect threats when they appear:

- Audit logging should be enabled
- A log monitoring and alerting system should be implemented

Security Never Sleeps: Here are some best practices to ensure your cluster stays a safe haven:

- Security patches and updates should be applied quickly
- Vulnerability scans and penetration tests should be done semi-regularly
- Any obsolete components in the cluster should be removed

Kubernetes Security Best Practices in Action:

RBAC: RBAC (Role-Based Access Control) in Kubernetes regulates access to a Kubernetes cluster and its resources based on defined roles and permissions. These permissions (permission to create/delete x resource, etc.) are assigned to users, groups or service accounts. RBAC is a good way to ensure the resources in your cluster can only be accessed by those who **need** to access it. RBAC can be configured using a YAML file (same as defining a resource) where specific rules can be defined by declaring the resource type and verbs. Verbs are the actions being restricted, such as 'create' and 'get'.

Secrets Management: Kubernetes Secrets protect sensitive credentials through base64 encoding, requiring encryption-at-rest and strict RBAC controls to prevent unauthorized access and data leakage.

PSA (Pod Security Admission) and PSS (Pod Security Standards): Pod Security Standards are used to define security policies at 3 levels (privileged, baseline and restricted) at a namespace or cluster-wide level. What these levels mean:

- Privileged: This is a near unrestricted policy (allows for known privilege escalations)
- Baseline: This is a minimally restricted policy and will prevent known privilege escalations (allows deployment of pods with default configuration)
- Restricted: This heavily restricted policy follows the current pod hardening best practices

Pod Security Admission (using a Pod Security Admission controller) enforces these Pod Security Standards by intercepting API server requests and applying these policies.

Answer the questions below

Which best container security practice is used to regulate access to a Kubernetes cluster and its resources?

RBAC

What is used to define security policies at 3 levels?

Pod Security Standards

What enforces these policies?

Pod Security Admission

What Kubernetes object can be used to store sensitive information and should, therefore, be managed securely?

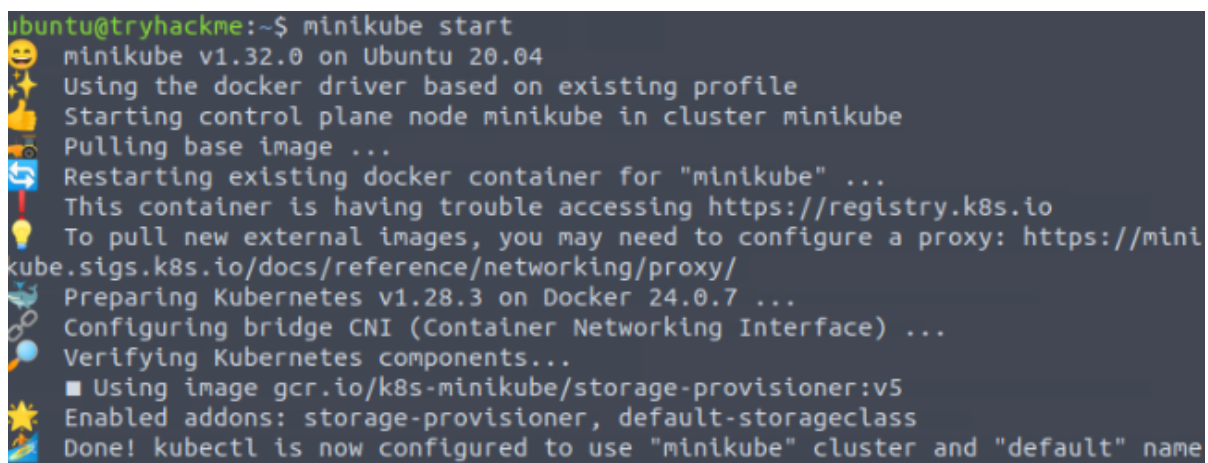
Secret

Task 8 - Hands-on with Kubernetes

Press the green **Start Machine** button located at the top-right of this task.

Phase One: Explore: Okay, let's get going, shall we? The first thing we want to do is start our Minikube cluster. You can do this by running the following command in the terminal (the cluster will take a couple of minutes to start up):

```
minikube start
```

A terminal window screenshot showing the output of the 'minikube start' command. The output includes: 'minikube v1.32.0 on Ubuntu 20.04', 'Using the docker driver based on existing profile', 'Starting control plane node minikube in cluster minikube', 'Pulling base image ...', 'Restarting existing docker container for "minikube" ...', 'This container is having trouble accessing https://registry.k8s.io', 'To pull new external images, you may need to configure a proxy: https://minikube.sigs.k8s.io/docs/reference/networking/proxy/', 'Preparing Kubernetes v1.28.3 on Docker 24.0.7 ...', 'Configuring bridge CNI (Container Networking Interface) ...', 'Verifying Kubernetes components...', 'Using image gcr.io/k8s-minikube/storage-provisioner:v5', 'Enabled addons: storage-provisioner, default-storageclass', and 'Done! kubectl is now configured to use "minikube" cluster and "default" namespace'.

Once the cluster has started up, you're ready to go! This guided walkthrough will take you through this cluster, reinforcing knowledge taught in the room. That being said, you have a Kubernetes cluster at your disposal, so feel free to explore the cluster and experiment with the different commands you have learned so far! Let's see what's running already, shall we? We can do this using the following command (`-A` for all namespaces)

```
kubectl get pods -A
```

After running this, you should see a few pods running. These are the default pods present when you first start a Kubernetes cluster (you may recognise some of these names as they represent some of the control plane and worker node processes). Exciting stuff! But how about we make it more exciting by adding a deployment and service into the mix? On the VM you will be able to find some config YAML files located here:

```
~/Desktop/configuration
```

There are two config YAMLs here of interest to us: **nginx-deployment.yaml** and **nginx-service.yaml**. Check these configuration files out using the cat command:

```
cat filename
```

Let's break down what we see in each of these files:

nginx-deployment.yaml : This is a relatively simple deployment; we can see the desired state has been defined as a single replica pod, inside of which will be a container running an nginx image. From the other lines, we can determine that this pod will be used to run some kind of web app.

nginx-service.yaml : This, again, is a straightforward nginx NodePort service being defined here. The eagle-eyed among you may have noticed that the 'selector: → app: ' field matches the app label defined in the deployment, as well as the 'targetPort' matching the 'containerPort' outlined in the deployment. This service exposes the web application running in the pod, which the deployment controls. To apply the configuration outlined in these YAML files, we use the kubectl apply command. Remember to apply the service file first! Like so:

```
kubectl apply -f nginx-service.yaml
```

```
kubectl apply -f nginx-deployment.yaml
```

Verify the replica pod is running by using the following command (not providing any namespace flag will get all the pods in the default namespace, which is where our pod should be):

```
kubectl get pods -A
```

```
ubuntu@tryhackme:~$ ls
Desktop  Downloads  Pictures  Templates  kubectl      minikube-linux-amd64
Documents Music      Public    Videos    kubectl.sha256 nginx.tar

ubuntu@tryhackme:~$ cd Desktop/configuration/
ubuntu@tryhackme:~/Desktop/configuration$ ls
nginx-deployment.yaml  nginx-service.yaml  rbac
ubuntu@tryhackme:~/Desktop/configuration$ kubectl apply -f nginx-service.yaml

service/nginx-service created
ubuntu@tryhackme:~/Desktop/configuration$
ubuntu@tryhackme:~/Desktop/configuration$ kubectl apply -f nginx-deployment.yaml
deployment.apps/nginx-deployment created
ubuntu@tryhackme:~/Desktop/configuration$ kubectl get pods -A
NAMESPACE      NAME                                                    READY   STATUS    RESTARTS
  AGE
default        nginx-deployment-66d6696594-hlqnr                     1/1     Running   0
  29s
kube-system    coredns-5dd5756b68-qbss6                             1/1     Running   1 (4m21s ago)
  189d
kube-system    etcd-minikube                                          1/1     Running   1 (4m21s ago)
  189d
kube-system    kube-apiserver-minikube                              1/1     Running   2 (2m37s ago)
  189d
kube-system    kube-controller-manager-minikube                     1/1     Running   3 (2m15s ago)
  189d
kube-system    kube-proxy-pb4vz                                       1/1     Running   1 (4m21s ago)
  189d
kube-system    kube-scheduler-minikube                              1/1     Running   1 (4m21s ago)
  189d
kube-system    storage-provisioner                                   1/1     Running   3 (61s ago)
  189d
ubuntu@tryhackme:~/Desktop/configuration$
```

You should now see a pod with 'nginx-deployment' in its name! Our deployment has been started!

Phase Two: Interact: Okay, now we have a web application running in a pod, which is being exposed by a service. If you recall the nginx-service.yaml, the service connects to the web application using the target port of port 80 (the port where the container is exposed). However, the service's port itself is set to 8080. We want to access this service. We will do this by using the kubectl port-forward command, which allows us to forward a port on our localhost to the Kubernetes service port (8080). Here is the full command to do so:

```
kubectl port-forward service/nginx-service 8090:8080
```

After running this command, open a web browser (Firefox) and access the web application at the following address:

```
http://localhost:8090/
```

Looks like a simple login terminal, but it needs credentials. Why don't we see if there are any Kubernetes secrets on the cluster that can help us log in to the terminal? Open another terminal window (so the previous window continues to port-forward) and run the following command to see if there are any secrets (in the default namespace):

```
kubectl get secrets
```

Ahh, there is! "terminal-creds" sounds like we are onto a winner! Using the `kubectl describe` command, we can get more details on this secret to see what is being stored here:

```
kubectl describe secret terminal-creds
```

In the description, we can see that two pieces of "Data" are being stored: a username and a password. While Kubernetes secrets **are** stored in plaintext and not encrypted by default, they are base64 encoded, so we pipe this command and base64 decode the output to get it in plain text. To access this data, we can use the following command:

To get the username, run:

```
kubectl get secret terminal-creds -o jsonpath='{.data.username}' | base64 --decode
```

To get the password, run:

```
kubectl get secret terminal-creds -o jsonpath='{.data.password}' | base64 --decode
```

```
d}' | base64 --decode
```

Use these credentials to access the login terminal. That's a bingo! We're in, and you have retrieved the flag!

Bonus Task: For those curious enough, you can use an alternate method to get this flag. It will require some Kubernetes investigation on your part, but the first breadcrumb lies in the `nginx-deployment.yaml`!

Phase Three: Secure: Time for us to get to work with some Kubernetes secret management. With these credentials being sensitive information, we want to restrict access to the Kubernetes secret they are stored in. We can do this by configuring RBAC (Role-Based Access Control).

First things first, let's decide who it is we want to be able to access this secret. Your DevSecOps manager has suggested that you restrict access to a service account, which is essentially an identity a pod can assume to interact with the Kubernetes API/cluster. By doing this, we can maybe even set it up so that in the future, our daily terminal tasks can be run by an application in a pod. Let's use the `kubectl create serviceaccount` (can be abbreviated to `'sa'`) command to make two service accounts, the `'terminal-user'` for non-admin terminal activities (should not have access to secret) and the `'terminal-admin'` for admin terminal activities (should have access to secret). Run these two commands to make those service accounts:

```
kubectl create sa terminal-user
```

```
kubectl create sa terminal-admin
```

With those service accounts created, it's time to restrict access to the `'terminal-creds'` secret so that only the `'terminal-admin'` service account can access it. We are going to do this by defining and applying two configurations. First of all, a Role YAML that defines the role and what it can do (get the `'terminal-creds'` secret). Then, a Role Binding YAML that binds the role we have defined to the `'terminal-admin'` service account. Navigate to the following directory and cat the two YAMLs to examine how these are defined:

```
~/Desktop/configuration/rbac
```

role.yaml : Here, you can see we define a role named "secret-admin". In the rules section, we define what it is this role can do. We define the resource (secrets), what verbs are being restricted (we are restricting the 'get' verb, but you could restrict others) and finally, the name of our secret (terminal-creds).

role-binding.yaml : In this YAML, we bind the 'terminal-admin' service account (in the 'subjects' section) with the 'secret-admin' role defined above (in the roleRef section).

Lets now apply these configurations the same way we applied the deployment and service (using kubectl apply):

```
kubectl apply -f role.yaml
```

```
kubectl apply -f role-binding.yaml
```

You have now configured RBAC for this Kubernetes secret! The only thing left to do is test whether our RBAC is working. We can do this using the kubectl auth command, which tells us if a service account has sufficient permission to perform a specific action. Let us first verify that the regular 'terminal-user' service account CAN NOT access the secret:

```
kubectl auth can-i get secret/terminal-creds --as=system:serviceaccount:default:terminal-user
```

It looks like we expected. This "no" response confirms that this service account can no longer access the terminal-creds secret. Now, finally, let us verify that our 'terminal-admin' service account CAN access it:

```
kubectl auth can-i get secret/terminal-creds --as=system:serviceaccount:default:terminal-admin
```

With this "yes" output, you have confirmed RBAC is in place and fulfilled your duty as a DevSecOps engineer, fortifying the cluster and taking a good first step into

hardening this cluster. Hope you've enjoyed taking a little tour around this Kubernetes cluster and getting to know the basics. Until next time!

Answer the questions below

| Can you master the basics of Kubernetes and retrieve the flag?

THM{k8s_k3nno1ssarus}

| What apiVersion is used for the RoleBinding?

rbac.authorization.k8s.io/v1